CyberLens

A Local Cyber Threat Intelligence and Incident Response Workspace

MINOR PROJECT REPORT

Submitted by

[Student Name]

[Register Number]

Under the guidance of

[Supervisor Name]

[Designation]

in partial fulfillment of the requirements for the degree of

[Degree Program]

[Department Name]

[College or Institute Name]

[City, State]

[Month Year]

DECLARATION

I hereby declare that this minor project report entitled “CyberLens: A Local Cyber Threat Intelligence and Incident Response Workspace” describes the project work submitted by me in partial fulfillment of the requirements for [Degree Program] at [College or Institute Name]. The implementation and written report are presented for academic evaluation. Any external standards, provider documentation, and software documentation used to explain the design are cited in the references.

I understand that the institution’s own declaration and originality requirements take precedence. I will review this report, replace the editable student and institution details, verify the implementation claims against my final project build, and sign this statement only if it is accurate.

Student Details	To be completed
Name	[Student Name]
Register number	[Register Number]
Signature	_____
Date	_____

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “CyberLens: A Local Cyber Threat Intelligence and Incident Response Workspace” is submitted by [Student Name], [Register Number], in partial fulfillment of the requirements for [Degree Program] at [College or Institute Name]. The project work was carried out under the guidance of [Supervisor Name], [Designation], [Department Name].

The final wording, institutional seal, signatures, and date must be completed by the authorized supervisor and department in accordance with the institution’s report format.

Role	Name and signature	Date
Project supervisor	[Supervisor Name] _____	_____
Head of department	[HOD Name] _____	_____
External examiner	_____	_____

ACKNOWLEDGEMENTS

I express my sincere gratitude to [Supervisor Name] for guidance, feedback, and support during the development of CyberLens. The discussions helped shape the project scope and kept the implementation focused on evidence review, analyst decisions, and safe investigation workflows.

I thank the faculty and staff of [Department Name], [College or Institute Name], for providing the academic environment and resources required to complete this project. I also acknowledge the maintainers of the open-source software and public cybersecurity data sources cited in this report.

Finally, I thank my family and peers for their encouragement throughout the project.

[Student Name]

ABSTRACT

CyberLens is a browser-based workspace for organizing cyber threat intelligence, security observations, and incident-response records for assets that an analyst is authorized to assess. It combines an inventory of domains, IP addresses, URLs, email addresses, applications, and software components with background investigations that collect selected evidence from configured sources. Each investigation records collector status, observation time, source provenance, and evidence checksums. Vulnerability matches remain review candidates until an analyst evaluates applicability, while a documented academic prioritization method explains which reviewed evidence contributes to risk.

The application uses a React and TypeScript interface, a FastAPI service, PostgreSQL for durable records, Redis with RQ for background processing, a separate dispatcher for reliable queue delivery, and Docker Compose for local operation. Analysts can review evidence relationships, compare investigation history, create incident cases, track remediation plans, and export PDF or JSON report snapshots. Security controls include role-based access, cookie-backed sessions, CSRF checks, password hashing, shared login throttling, scoped authorization records, bounded outbound requests, and backend-only provider credentials.

CyberLens has a separate offline Demo mode that uses synthetic fixtures. The normal Live mode can use real provider data when the required configuration, access, and authorization are available. The project is designed as a local single-workspace showcase and does not perform exploitation, network-wide scanning, automatic containment, or automatic remediation. The risk score is an explainable academic prioritization aid and has not been calibrated against incident outcomes.

Keywords: cyber threat intelligence, incident response, evidence provenance, vulnerability applicability, risk prioritization, background jobs

TABLE OF CONTENTS

ABSTRACT .....

ABBREVIATIONS .....

CHAPTER 1 INTRODUCTION .....

1.1 Background .....

1.2 Problem Statement .....

1.3 Aim and Objectives .....

1.4 Scope and Boundaries .....

1.5 Project Contributions .....

CHAPTER 2 TECHNICAL BACKGROUND AND RELATED WORK .....

2.1 Cyber Threat Intelligence and Asset Context .....

2.2 Vulnerability Sources and Prioritization Signals .....

2.3 ATT&CK and Structured Threat Knowledge .....

2.4 Background Processing and Evidence .....

2.5 Design Opportunity .....

CHAPTER 3 REQUIREMENTS AND SYSTEM DESIGN .....

3.1 Requirements .....

3.2 Architecture .....

3.3 Investigation Workflow .....

3.4 Data Model .....

3.5 Risk Assessment Method .....

3.6 Security and Privacy Design .....

3.7 Operating Modes .....

CHAPTER 4 IMPLEMENTATION AND EVALUATION .....

4.1 Technology Stack .....

4.2 Core Modules .....

4.3 Durable Investigation Orchestration .....

4.4 Evidence, Findings and Change Radar .....

4.5 Case Management and Reports .....

4.6 Verification and Observed Results .....

4.7 Limitations .....

CHAPTER 5 CONCLUSION AND FUTURE WORK .....

5.1 Conclusion .....

5.2 Future Work .....

REFERENCES .....

APPENDIX A API RESOURCE SUMMARY .....

APPENDIX B LOCAL RUN AND SHOWCASE NOTES .....

LIST OF FIGURES

Fig. 3.1 CyberLens service architecture .....
Fig. 3.2 Investigation processing and analyst review workflow.....
Fig. 3.3 Core data relationships .....

LIST OF TABLES

Table 3.1 Functional requirements .....
Table 3.2 Non-functional requirements .....
Table 3.3 Core data entities .....
Table 4.1 Technology stack .....
Table 4.2 Selected evidence sources and interpretation .....
Table 4.3 Documented verification evidence .....
Table 4.4 Limitations and proposed extensions .....

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
CISA	Cybersecurity and Infrastructure Security Agency
CPE	Common Platform Enumeration
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CSRF	Cross-Site Request Forgery
CTI	Cyber Threat Intelligence
DNS	Domain Name System
EPSS	Exploit Prediction Scoring System
HIBP	Have I Been Pwned
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
KEV	Known Exploited Vulnerabilities
NVD	National Vulnerability Database
OSV	Open Source Vulnerabilities
RQ	Redis Queue
SSRF	Server-Side Request Forgery
STIX	Structured Threat Information Expression
TLS	Transport Layer Security

CHAPTER 1

INTRODUCTION

1.1 Background

Organizations use public vulnerability databases, threat feeds, asset inventories, and incident records to understand security exposure. The information is distributed across providers and differs in scope, freshness, and confidence. A CVE record can describe a product weakness without proving that the affected version is installed. A threat-feed match can be stale or incomplete. A historical email-breach record does not establish current account compromise. Analysts therefore need a workspace that preserves what each source observed and makes uncertainty visible.

CyberLens brings asset context, selected source observations, evidence, findings, and response records into one local browser application. The system uses asynchronous investigations so the web request can return while the worker performs provider checks. The analyst reviews the stored results and determines how they apply to the organization.

1.2 Problem Statement

Security information often arrives as separate API responses, spreadsheets, and incident notes. When these records are not connected to the asset and the observation that produced them, analysts can lose source provenance, repeat work, or overstate what a result means. A vulnerability candidate may be treated as present without checking the installed version. A provider outage may be mistaken for an absence of risk. Risk scores can also appear authoritative even when the scoring method is not calibrated against real incident outcomes.

The project addresses the need for a single analyst workflow that records authorized assets, runs bounded investigations, preserves source status and evidence, separates candidate detection from applicability decisions, and supports case tracking and report generation without automating disruptive response actions.

1.3 Aim and Objectives

The aim of CyberLens is to provide a local workspace that helps analysts collect, review, and document security evidence for authorized assets.

- Provide inventory records for domains, IP addresses, URLs, email addresses, applications, and software components.

- Run selected Live-mode collectors as background work and record success, partial coverage, failure, freshness, and provenance.
- Preserve evidence snapshots and connect findings and relationships to their supporting evidence.
- Require analyst review of vulnerability applicability before candidate findings affect vulnerability risk.
- Explain the project's academic prioritization score and preserve changes as assessment revisions.
- Support incidents, response tasks, remediation plans, control mappings, monitoring history, and saved PDF or JSON reports.
- Use role-based authorization, CSRF protection, session management, audit records, and bounded outbound networking.
- Provide an isolated offline Demo mode for repeatable workflows without real provider requests.

1.4 Scope and Boundaries

CyberLens supports analyst-led assessment and incident documentation. Depending on configuration and provider access, collectors can use DNS and RDAP context, bounded website and TLS observations, existing VirusTotal reports, explicit email-breach lookups, vulnerability information from NVD and OSV, CISA KEV and FIRST EPSS context, and locally matched threat-feed snapshots. Optional community sources have separate consent or permitted-use restrictions. The Integrations view reports source status; a configured API key does not guarantee quota, availability, or a useful result.

The system does not exploit vulnerabilities, guess passwords, scan arbitrary network ranges, automatically block traffic, disable accounts, remediate software, or send external incident notifications. Website checks are bounded and require a current authorization record. Live mode is a local single-workspace operation profile, not a public internet production deployment.

1.5 Project Contributions

The project combines provider observations with a durable evidence model and analyst workflow. Its main contribution is the connection between each finding or relationship and the saved evidence that supports it. Collector failure and missing coverage are represented explicitly so the interface can distinguish unknown from no match.

A transactional outbox and separate dispatcher reduce the chance that an investigation is saved in PostgreSQL but never delivered to the worker queue. Stable job identifiers

make repeated delivery idempotent. The workflow also keeps technical evidence separate from analyst decisions, response tasks, remediation plans, and immutable report snapshots.

1.6 Organization of the Report

Chapter 1 introduces the project and its scope. Chapter 2 describes the technical background and source models. Chapter 3 presents requirements, architecture, data relationships, risk methodology, and security design. Chapter 4 explains the implementation and documented verification evidence. Chapter 5 concludes the report and identifies future work.

CHAPTER 2

TECHNICAL BACKGROUND AND RELATED WORK

2.1 Cyber Threat Intelligence and Asset Context

Cyber threat intelligence (CTI) is useful when an observation can be connected to a specific asset and interpreted with its source, time, and limitations. CyberLens treats provider data as evidence to review, rather than as a command to act. A finding retains its source attribution and observation context. The system's evidence graph projects stored relationships from PostgreSQL records and links each edge to the evidence snapshot that supports it.

Asset context matters because a provider match alone does not determine organizational impact. The same CVE may be irrelevant to an asset that does not run the affected component, while a lower-severity observation may matter for a critical exposed service. CyberLens records inventory attributes and analyst applicability decisions to keep these questions visible.

2.2 Vulnerability Sources and Prioritization Signals

The National Vulnerability Database (NVD) provides machine-readable vulnerability and product data through its API. CyberLens uses CPE and CVE information to surface candidate matches for inventoried software. Product-name and version matching can be ambiguous, so a returned candidate remains in a review state until an analyst decides whether it applies [7].

CISA's Known Exploited Vulnerabilities catalog identifies vulnerabilities that CISA lists as known exploited [8]. FIRST EPSS provides a separate estimate of exploitation probability over the next 30 days [9]. These signals answer different questions: CVSS describes technical severity, KEV indicates catalog membership, and EPSS estimates observed exploitation likelihood at a population level. CyberLens stores and displays these concepts separately; none proves that a specific asset has been compromised.

For package-level analysis, CyberLens can query OSV for an explicitly inventoried npm or PyPI package and version. It does not upload a software bill of materials or discover dependencies automatically. A package advisory is a candidate, and aliases are retained for provenance while duplicate scoring against an NVD CVE is avoided.

2.3 ATT&CK and Structured Threat Knowledge

MITRE ATT&CK organizes adversary tactics and techniques based on observed behavior. CyberLens stores a locally versioned Enterprise subset and supports an administrator-queued synchronization of official STIX 2.1 data outside Demo mode [10]. A technique suggestion is contextual and requires analyst review. It does not establish that an attacker used the technique.

Versioned catalog snapshots help preserve the meaning of existing mappings when the upstream knowledge base changes. A mapping keeps its catalog version and saved technique text. Existing reports are snapshots and are not silently rewritten when a newer catalog is activated.

2.4 Background Processing and Evidence

Provider requests can be slow, rate-limited, or temporarily unavailable. A background queue separates the user-facing API from the work performed by collectors. RQ represents Python functions as jobs executed by worker processes using Redis [4, 5]. CyberLens adds a PostgreSQL outbox and dispatcher around this queue so the accepted investigation and its delivery event are recorded together.

Evidence provenance includes the source, provider or version where available, observation time, freshness, status, schema version, and a checksum. Hashes help detect changes to stored bytes. They do not independently prove collection authenticity or establish forensic chain of custody.

2.5 Design Opportunity

The design opportunity is to make the analyst workflow as important as the external lookup. CyberLens joins source collection to inventory, evidence, applicability review, prioritization, incident management, remediation tracking, and report snapshots. This approach is intended to reduce context switching and preserve uncertainty. It does not replace the source providers, a dedicated vulnerability scanner, a SIEM, or a calibrated enterprise risk model.

CHAPTER 3

REQUIREMENTS AND SYSTEM DESIGN

3.1 Requirements

The system requirements focus on a local, authenticated analyst workspace with durable records and controlled provider interaction. Functional requirements describe what the workspace supports; non-functional requirements describe the reliability, safety, and usability expected from those workflows.

Table 3.1. Functional requirements

ID	Requirement	Expected behavior
FR-01	Asset inventory	Create and maintain scoped domain, IP, URL, email, application, and component records.
FR-02	Investigation	Queue an investigation and show its attempts, collectors, evidence, and findings.
FR-03	Analyst review	Record applicability decisions with a reason and preserve review history.
FR-04	Incident response	Create cases, events, playbook tasks, remediation actions, and control mappings.
FR-05	Reporting	Save immutable PDF or JSON snapshots from current workspace records.
FR-06	Monitoring	Schedule authorized website checks and compare saved investigation history.
FR-07	Administration	Manage roles and inspect audit, integrity, provider, and health status.

Table 3.2. Non-functional requirements

Area	Requirement
Security	Protect credentials, enforce role checks and CSRF validation, and restrict outbound network requests.
Reliability	Persist investigation delivery events, use stable job identifiers, and separate attempts from the parent investigation.
Traceability	Attach evidence, source status, observation time, and immutable report snapshots to analyst decisions.
Usability	Present unavailable, stale, unassessed, no-match, and synthetic states distinctly.
Portability	Run the application components together using Docker Compose on a local machine.
Privacy	Keep provider secrets on the backend and require explicit, scoped authorization for protected lookups.

3.2 Architecture

The browser serves as the analyst interface. Caddy terminates local HTTPS and routes requests to the static frontend and API. The FastAPI service validates requests and persists operational records to PostgreSQL. When an investigation is accepted, the API also records an outbox event in the same transaction. The dispatcher publishes pending events to RQ in Redis, and a worker executes collectors and persists the results.

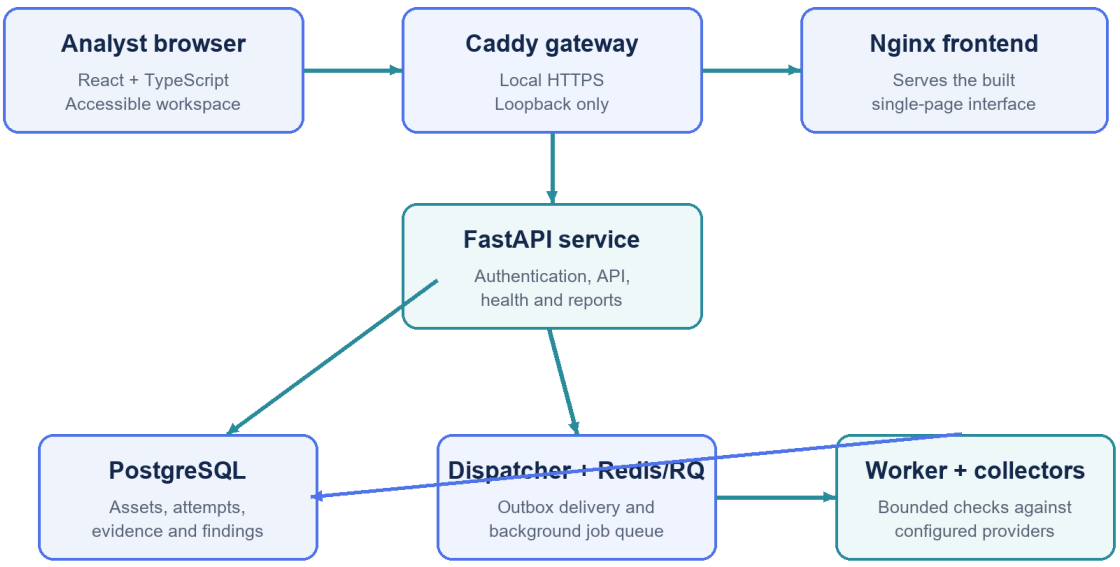


Fig. 3.1. CyberLens service architecture.

PostgreSQL is the durable source of truth for assets, investigations, evidence, findings, risk revisions, incidents, remediation, reports, and audit records. Redis is used for job delivery and shared short-lived state such as throttling and worker heartbeat. The evidence graph is a projection of the stored relational records; the application does not depend on a separate graph database.

3.3 Investigation Workflow

The workflow begins when an authenticated Analyst or Admin submits an investigation for an asset in scope. The API stores the investigation, its first attempt, and a delivery event transactionally. The dispatcher publishes a stable job identifier to the Redis-backed RQ queue. A worker claims the attempt, checks its lease and any required authorization, executes the selected collectors, then saves collector status, evidence snapshots, findings, and risk assessment revisions. The frontend reads the persisted state and updates the investigation view.



Fig. 3.2. Investigation processing and analyst review workflow.

A transient provider failure can receive a limited automatic retry. Each retry is a distinct attempt so the record shows what happened on each execution. Stable job identifiers make delivery idempotent; a reconciler can recover old queued or stale work. A failed collector produces unknown coverage rather than evidence that the asset is safe.

3.4 Data Model

The schema uses relational entities with explicit identifiers and foreign keys. Users and sessions represent authenticated access. Assets define the scope of investigations. An investigation has attempts and collector runs; each collector run can create evidence snapshots and findings. Risk assessments preserve a current projection, while risk revision records retain the earlier score explanations. Incidents link findings and response events. Remediation plans, actions, and control mappings track analyst work. Reports store point-in-time content. Audit events record important changes.

Table 3.3. Core data entities

Entity group	Purpose
Users, sessions, audit	Identity, roles, session lifecycle, and append-only activity history.
Assets and authorizations	Inventory context and time-limited permission for website or email checks.

Entity group	Purpose
Investigations and attempts	Parent work item, retries, transactional outbox delivery, and collector execution.
Evidence and relationships	Source-backed snapshots, hashes, typed graph edges, observation time, and coverage.
Findings and risk revisions	Reviewable candidates, applicability, score inputs, and historical explanations.
Incidents and remediation	Cases, timeline events, playbook tasks, plans, actions, and control mappings.
Reports and catalogs	Saved report snapshots, versioned ATT&CK data, and feed snapshots.

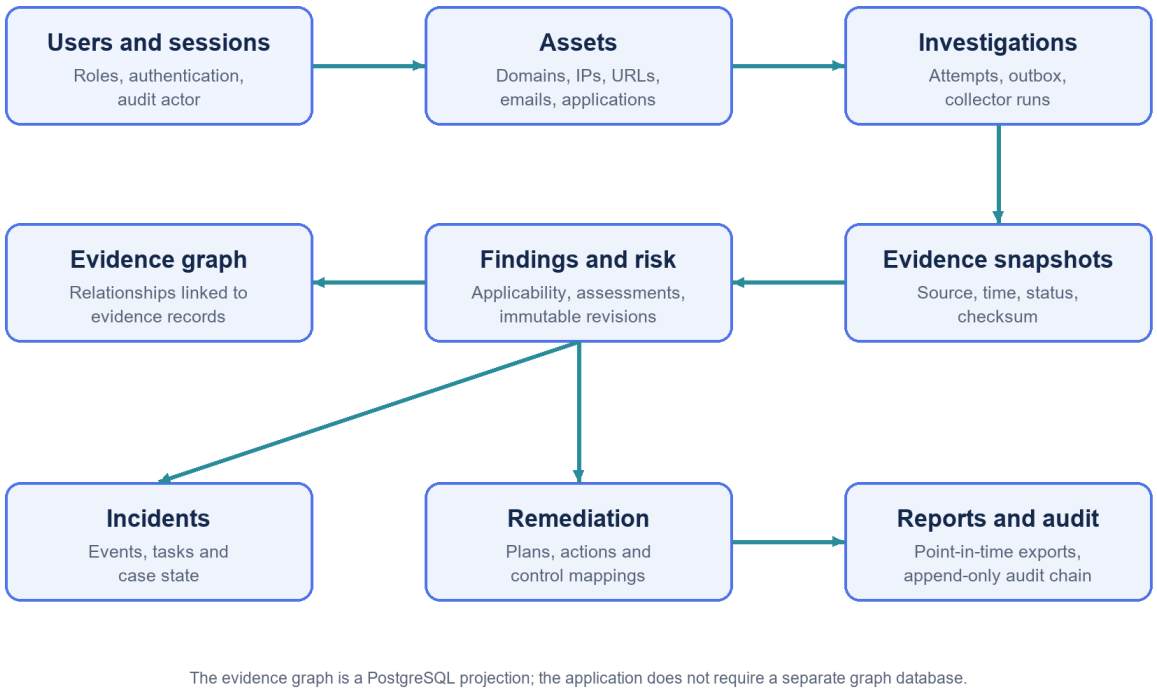


Fig. 3.3. Core data relationships.

3.5 Risk Assessment Method

CyberLens uses the documented policy named cyberlens-academic-1.0 to produce a deterministic prioritization score from assessed evidence categories. It records the observations, category coverage, exclusions, applicability decisions, and score revision so an analyst can inspect why a value changed. Unavailable or missing sources reduce coverage and are not silently converted into evidence of safety.

A vulnerability candidate begins as needs_review. Only an analyst decision that it applies contributes to vulnerability risk. The project also treats an applicable KEV item with a severity floor when required by the policy. EPSS is stored separately and does not affect this score. The score is an academic prioritization method and has not been calibrated using a labeled set of incident outcomes.

3.6 Security and Privacy Design

The workspace uses role-based authorization for Admin, Analyst, and Viewer roles. Passwords are stored as Argon2 hashes. Session cookies are protected with HTTP-only settings, CSRF tokens are required for state-changing requests, and Redis provides shared login throttling. Administrative password resets revoke the target user's active sessions and require a password change at next sign-in.

Provider keys stay on the backend. Authenticated provider requests are restricted to the provider's intended origin. Outbound HTTP requests use fixed or validated destinations, public-address checks, DNS pinning, redirect validation, byte limits, and request deadlines to reduce server-side request forgery exposure [12]. Website inspections require active authorization and are bounded observations; they do not exploit or enumerate a target.

The application records evidence checksums and an append-only audit hash chain. These controls help detect later changes to stored content. They do not prove who collected evidence or establish a complete forensic chain of custody. New passwords in Live mode are screened using the keyless HIBP Pwned Passwords range protocol; explicit email-breach searches are a separate provider capability and may require an HIBP subscription key [11].

3.7 Operating Modes

The default Docker Compose project uses Live mode. It can make real provider requests when each integration is enabled, credentials and permission are available, and the source is reachable. Results can be partial, stale, unavailable, or quota-limited. The user interface retains these states.

The separate Demo Compose project uses synthetic fixtures and makes no provider or website requests. It has separate named PostgreSQL and Redis volumes. Live and Demo both use the local gateway port 8443, so only one gateway should run on that host port at a time. Live mode is a local showcase configuration; it is not a hardened public deployment.

CHAPTER 4

IMPLEMENTATION AND EVALUATION

4.1 Technology Stack

CyberLens separates the browser interface, API, durable storage, queue, dispatcher, and worker into services. The frontend uses React and TypeScript for component-based screens [1]. Vite produces the static build, and Nginx serves it. FastAPI provides typed HTTP endpoints and OpenAPI documentation [2]. SQLAlchemy maps application records to PostgreSQL, Alembic applies schema migrations, and psycopg connects the Python service to PostgreSQL [3]. Redis and RQ support background job execution [4, 5]. Docker Compose defines the local service stack [6].

Table 4.1. Technology stack

Component	Technology	Role in CyberLens
Frontend	React 18, TypeScript, Vite, Recharts	Analyst workspace, status views, graphs, and charts.
Static web server	Nginx	Serves the compiled frontend inside its container.
Gateway	Caddy	Local HTTPS, frontend routing, and API reverse proxy.
API	Python 3.12, FastAPI, Uvicorn	Authentication, validation, APIs, health checks, and reports.
Persistence	PostgreSQL 16, SQLAlchemy, Alembic	Durable relational data and versioned schema changes.
Queue and worker	Redis 7, RQ, Python worker	Background jobs, delivery state, and collector execution.
Local orchestration	Docker Compose	Builds and starts the application services and named volumes.

4.2 Core Modules

The asset module stores domains, IP addresses, URLs, email addresses, applications, and software components. Authorization records control website inspections and selected email-provider requests. The investigation module coordinates collectors and exposes status and history. The findings module stores provider-backed candidates and analyst applicability decisions. The risk view shows the score, contributors, assessed categories, excluded coverage, sensitivity, and historical revisions.

Incident management links cases to findings and investigations. A case contains status history, timeline events, and playbook tasks. Remediation plans and actions track proposed work and verification instructions. Control mappings retain a rationale and confidence. Reports capture PDF or JSON snapshots, while the integrity view checks evidence checksums and the audit chain.

Other modules include threat modeling, the evidence graph, Change Radar, continuous monitoring, provider status, posture scorecards, MITRE ATT&CK mappings, and shared threat-feed snapshots. These modules reuse the same authenticated API, persisted records, and audit model.

4.3 Durable Investigation Orchestration

The API does not perform all provider calls synchronously inside the user request. It commits the investigation, initial attempt, and outbox delivery event together. The separate dispatcher claims pending events and enqueues RQ jobs using stable identifiers. If delivery is repeated, the stable job ID prevents duplicate execution. The worker claims the attempt, refreshes its heartbeat, runs the selected collectors, and saves each collector run and evidence snapshot.

The design separates the authoritative business record from the transient queue. PostgreSQL retains the investigation and delivery intent; Redis carries the job to a worker. A reconciler recovers old queued deliveries, handles stale running attempts, and materializes due monitoring runs. Limited exponential backoff is used for transient provider or network failures. Permanent configuration or authorization failures do not become unbounded retries.

4.4 Evidence Sources and Interpretation

The collectors combine externally sourced observations with local matching against stored snapshots. A source result carries a status and provenance so the analyst can distinguish match, no match, partial coverage, stale data, provider outage, and not-applicable conditions.

Table 4.2. Selected evidence sources and interpretation

Source or collector	CyberLens use	Interpretation limit
DNS and RDAP	Collects domain and registration context.	Public registration and DNS data may be incomplete or privacy-protected.
Website and TLS	Observes a bounded HTTP response, selected headers, redirects, and certificate details.	One observation is not a complete web or TLS audit.
VirusTotal	Reads a provider report for an explicitly submitted asset when configured.	Provider quota and verdicts do not prove compromise.
NVD and CPE	Returns vulnerability candidates based on inventoried product/version context.	A candidate requires analyst confirmation of applicability.
CISA KEV and FIRST EPSS	Adds catalog membership and a separate population-level exploitation estimate.	Neither signal proves exploitation of the specific asset.
OSV	Checks an explicit npm or PyPI package and version.	No automatic dependency discovery or SBOM upload occurs.
Threat-feed snapshots	Matches an explicitly submitted public IP or URL	A no-match means only absence from that

Source or collector	CyberLens use	Interpretation limit
	against a validated local snapshot.	snapshot.
HIBP or XposedOrNot	Performs explicitly selected email-breach lookups under provider-specific access and consent.	Historical exposure does not mean the account is currently compromised.

NVD, CISA KEV, FIRST EPSS, and MITRE ATT&CK provide distinct data models and should not be merged into a single unqualified verdict [7-10]. The application presents provenance and limitations alongside observations. A source outage leaves prior evidence unknown unless a later successful collection can support a meaningful comparison.

4.5 Evidence Graph, Change Radar, and Reports

The evidence graph is built from stored assets, domains, IP addresses, certificates, software, CVEs, findings, investigations, incidents, and tasks. Each relationship stores a foreign key to supporting evidence, its source and collector, observation time, and whether it is direct evidence or contextual linkage. The graph is a projection from PostgreSQL data and is not a separate source of truth.

Change Radar compares an investigation with a prior saved run and classifies relationships as new, changed, no longer observed, or unknown. A failed, partial, unavailable, not-applicable, or unrun collector cannot justify a removal classification. Reports store a point-in-time snapshot so later changes to a case or remediation plan do not silently rewrite an earlier report.

4.6 Verification and Observed Results

The repository’s engineering record dated 26 October 2026 documents frontend TypeScript checks and a production build, a combined workflow and presentation suite with 78 passing tests in an isolated offline backend plus mocked-provider setup, and a separate real-backend smoke test with one passing test. The same record states that no live-provider test was run. These checks support the reported local workflow and interface behavior; they do not establish current external provider availability or public-production readiness.

The verification record also documents role and workflow coverage, responsive views, keyboard/accessibility checks, reports, monitoring, integrity, integrations, and persistence across a full-service restart in earlier isolated runs. The report does not claim these historical checks were rerun during report creation. A reviewer can reproduce the application workflow locally with the project Compose instructions and inspect the health endpoints.

Table 4.3. Documented verification evidence

Verification item	Recorded result	Scope
Frontend TypeScript and production build	Passed in the 26 Oct 2026 UI review.	Application and Playwright TypeScript; frontend bundle.
Offline workflow and mocked presentation suite	78 passed in 5.2 minutes.	Isolated offline backend workflows and mocked UI/provider cases.
Real-backend smoke test	1 passed in 7.6 seconds.	Isolated local backend login and dashboard; not a live provider check.
Live provider validation	Not run in the cited review.	External provider behavior remains environment-dependent.

4.7 Limitations

Live results depend on network access, provider terms, account tier, API quota, freshness, and source quality. Some integrations are optional or disabled by default. A provider response is an observation from that provider and not a complete security assessment.

The risk method is deterministic and explainable but is not calibrated against a labeled dataset of real incidents. The evidence checksum detects later byte changes but is not proof of forensic authenticity. The website collector performs a bounded observation rather than a comprehensive scan. OSV checks only explicitly inventoried package names and versions. Live and Demo each use a local single-workspace profile, and the current Compose deployment is not intended to be exposed publicly.

The optional evidence-grounded AI briefing is disabled in the current project configuration. CyberLens does not send evidence or analyst notes to an external AI service. Automatic remediation, containment, and external notifications are outside the project scope.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

CyberLens implements a local analyst workspace that connects asset inventory, background investigations, provider observations, evidence snapshots, findings, risk revisions, incident records, remediation tracking, and saved reports. Its architecture separates the browser interface from the API and long-running collection work. PostgreSQL preserves durable records, while the transactional outbox, dispatcher, Redis/RQ queue, and worker support reliable asynchronous processing.

The project emphasizes traceability and careful interpretation. Each finding can be reviewed against its evidence. Vulnerability applicability remains an analyst decision. Provider gaps are visible as unknown or partial coverage. Reports preserve the state captured when they were generated. These design choices help reduce overstatement and make the analyst's reasoning easier to inspect.

The project provides a Live mode that can use real sources when configured, and a separate offline Demo mode for repeatable synthetic workflows. The implementation is suitable for local demonstration and academic evaluation. Further operational hardening and live-provider testing would be required before any internet-facing deployment.

5.2 Future Work

Future work can improve both evaluation and deployment. The prioritization method could be calibrated and validated against a clearly defined incident or vulnerability outcome dataset. Additional integration tests could exercise external providers in a controlled account, with explicit quotas and reproducible fixtures.

A hardened deployment profile could add managed secrets, TLS certificate lifecycle, network segmentation, centralized backups, monitoring, disaster recovery, and multi-workspace isolation. The current application is a single-workspace local service, so tenant boundaries and operational controls would need careful design before shared deployment.

Other extensions include broader package ecosystem coverage, standardized import and export formats, richer evidence retention controls, additional role workflows, and improved source freshness diagnostics. Any automated response capability should require

explicit policy, authorization, auditability, and human approval rather than being inferred from a finding.

Table 4.4. Limitations and proposed extensions

Current limitation	Possible future work
Academic score is not calibrated to outcomes.	Develop a documented validation dataset and evaluate calibration and usefulness.
Live providers were not exercised in the cited UI verification.	Add controlled provider integration checks with quotas and consent.
Local single-workspace deployment.	Design and review tenant isolation, managed secrets, backups, and monitoring.
Package checks require explicit inventory.	Add authorized inventory import and additional package ecosystems.
No automated containment or remediation.	If added, require bounded policies, approval, audit, and rollback.

REFERENCES

[1] React Team, “Quick Start,” React Documentation. Available: https://react.dev/learn. Accessed 29 Oct. 2026.

[2] FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/. Accessed 29 Oct. 2026.

[3] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation.” Available: https://www.postgresql.org/docs/16/. Accessed 29 Oct. 2026.

[4] Redis, “Redis Job Queue,” Redis Documentation. Available: https://redis.io/docs/latest/develop/use-cases/job-queue/. Accessed 29 Oct. 2026.

[5] Python RQ, “Documentation Overview.” Available: https://python-rq.org/docs/. Accessed 29 Oct. 2026.

[6] Docker, “Docker Compose.” Available: https://docs.docker.com/compose/. Accessed 29 Oct. 2026.

[7] National Institute of Standards and Technology, “NVD API: Vulnerabilities.” Available: https://nvd.nist.gov/developers/vulnerabilities. Accessed 29 Oct. 2026.

[8] Cybersecurity and Infrastructure Security Agency, “Known Exploited Vulnerabilities Catalog.” Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog. Accessed 29 Oct. 2026.

[9] FIRST, “Exploit Prediction Scoring System (EPSS).” Available: https://www.first.org/epss/. Accessed 29 Oct. 2026.

[10] MITRE, “ATT&CK STIX Data.” Available: https://github.com/mitre-attack/attack-stix-data. Accessed 29 Oct. 2026.

[11] Have I Been Pwned, “API Documentation: Pwned Passwords and Breach Search.” Available: https://haveibeenpwned.com/API/v3. Accessed 29 Oct. 2026.

[12] OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet.” Available: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html. Accessed 29 Oct. 2026.

APPENDIX A API RESOURCE SUMMARY

The HTTP API is organized under /api/v1. The following resource groups summarize the workspace interface. The interactive OpenAPI reference is served at /docs when the local application is running.

Resource group	Representative operations
Authentication	CSRF token, sign in, current user, sign out, password change.
Assets	Create, read, update, authorize, and manage components for inventoried assets.
Investigations	Create, list, inspect, retry, compare, inspect evidence graph and changes.
Findings	Inspect candidate findings and record applicability decisions.
Incidents	Create and update cases, add timeline events, complete playbook tasks.
Reports	Create, retrieve, and download PDF or JSON snapshots.
Remediation	Create plans and actions; map findings to controls with rationale.
Integrity and operations	Check audit/evidence integrity, liveness, readiness, and metrics.

Mutating requests require the normal authenticated session and CSRF protection. Admin-only actions include user administration, catalog synchronization, and selected provider-feed refreshes. The API enforces roles on the server; hiding a control in the interface is not treated as an authorization boundary.

APPENDIX B LOCAL RUN AND SHOWCASE NOTES

The default local Compose project runs Live mode. Docker Desktop must be running before starting the stack. The gateway is bound to loopback on port 8443; the API, worker, dispatcher, PostgreSQL, and Redis are not published directly to the host.

Table B.1. Local service startup

Task	PowerShell command
Start the existing Live stack	docker compose up -d
Rebuild and start after source changes	docker compose up --build -d
Check service status	docker compose ps
Follow API logs	docker compose logs -f api
Open the workspace	https://localhost:8443

For an academic showcase, demonstrate an asset that is owned by the project team or explicitly authorized, then show the investigation status, source provenance, evidence, analyst applicability decision, score explanation, case or remediation record, and report snapshot. Provider status and quota should be checked before the presentation. Synthetic Demo results must be identified as synthetic.

Keep credentials and API keys in the local environment configuration. Do not include them in this report, screenshots, source control, or presentation slides. The report contains editable fields for student, supervisor, department, institute, degree program, and date; replace them with accurate institutional details before submission.